

AI Semester Project: **AI Mental Health** **Companion**



Submitted to Ms Warda Aslam

Submitted By Group Members:

Ghulam Qadir (230970)

Noor Malik (230964)

Semester 6

Section: BSIT-6-B

Contents

PROJECT REPORT: AI-POWERED MENTAL HEALTH COMPANION 1

1. Introduction 1

2. Problem Statement 1

 I. Data Isolation and Reactive Engineering 1

 II. Absolute Prevalence of Hardcoded Rule Engines 2

 III. System Latency and Monolithic Bottlenecks 2

 IV. Lack of Secure, Dynamic Telemetry Visualizations 2

3. Core Architectural Framework & Technical Specifications 2

 A. The Server-Side Blueprint (FastAPI Runtime Engine) 2

 B. The Client-Side Blueprint (React + Vite Canvas) 3

4. System Architecture and Diagrams 3

 A. Client-Side Layer (React Dashboard Canvas) 4

 B. Server-Side Layer (FastAPI Microservice Engine)..... 4

 C. The Machine Learning Dimensions (Dual Engine Strategy) 5

 D. Cloud Data Layer (Google Cloud Firebase Firestore)..... 6

5. Dataset, Preprocessing, and Exploratory Data Analysis (EDA)..... 6

 A. Dataset Selection & Feature Profiling..... 6

 B. Algorithmic Data Preprocessing Pipeline..... 7

 I. Missing Data Handling (Imputation Engine) 7

 II. Feature Scaling and Normalization 7

 C. Exploratory Data Analysis (EDA) Insights 8

6. Model Training, Comparative Evaluation, and Performance Metrics..... 9

 A. Mathematical and Algorithmic Foundation of the Models 9

 Model A: Random Forest Classifier (Ensemble Paradigm)..... 9

 Model B: Logistic Regression (Generalized Linear Paradigm) 9

 B. Comparative Performance Analysis & Matrix Evaluation..... 10

 C. Deep Architectural Insights & Confusion Matrix Mapping 10

 D. Operational Validation via ROC-AUC Curve..... 11

7. Evaluation Metrics & Charts (Diagnostic Stability Analysis) 11

 A. Analysis of Fig 6.1: Confusion Matrix Grid (Error Mitigation Proof) 12

 I. Random Forest Classifier (Selected Engine - Panel A)..... 12

 II. Logistic Regression (Baseline - Panel B) 12

 B. Analysis of Fig 6.2: ROC-AUC Curve (Operational Stability) 13

8. Frontend Screenshots..... 14

PROJECT REPORT: AI-POWERED MENTAL HEALTH COMPANION

1. Introduction

The contemporary healthcare landscape is experiencing an unprecedented paradigm shift, heavily driven by the integration of Artificial Intelligence (AI) and Machine Learning (ML) pipelines into daily clinical diagnostic and monitoring tasks. Among these dimensions, psychological wellness and scalable mental health tracking have emerged as critical areas requiring high-fidelity automated tools. Traditional mental health monitoring methodologies are historically constrained by intermittent clinical check-ins, subjective diagnostic self-reporting, and an absolute absence of real-time predictive telemetry that can alert care providers before an acute psychological crisis escalates.

To address these vulnerabilities, this project introduces the **AI-Powered Mental Health Companion Dashboard**. Engineered as a high-performance, completely decoupled full-stack digital health ecosystem, the system bridges the gap between reactive psychiatric consultation and proactive algorithmic analysis. The application core consists of a multi-tiered architecture: an asynchronous, high-speed Python **FastAPI** backend that exposes complex predictive modeling engines, seamlessly coupled with a responsive, client-side single-page application (SPA) authored in **React via the Vite building pipeline**.

The runtime infrastructure relies entirely on mathematical validation rather than static, hardcoded logic workflows. The system maps dual-engine diagnostic data vectors: a predictive numerical **Risk Assessment Profile Engine** coupled with a deep learning-based **Natural Language Text Semantic Journaling Engine**. By standardizing continuous, dynamic feedback loops, the platform converts qualitative emotional metrics into structured, quantitative historical logs stored instantly within **Google Cloud Firebase Firestore**. The end product is a high-availability dashboard tailored to provide instant analytical visual insights for both endpoints—the patient and the supervising clinical supervisor.

2. Problem Statement

Despite the proliferation of digital health trackers, current operational software frameworks exhibit profound systemic failure modes regarding automated mental health diagnostics and behavioral tracking. These core structural deficiencies include:

I. Data Isolation and Reactive Engineering

Existing mental health applications function as static, passive record-keeping diaries. They lack live, real-time automated processing loops. In typical configurations, a patient inputs psychological stressors, but the metrics remain unparsed inside localized tables. The data is only reviewed manually during the next scheduled medical consultation. This reactive framework is structurally incapable of continuous hazard profiling, leaving critical high-risk shifts unflagged for weeks at a time.

II. Absolute Prevalence of Hardcoded Rule Engines

A significant percentage of legacy mental health tracking tools rely on shallow, hardcoded threshold rule chains (e.g., standard `if/else` statements matching exact word tokens or basic tally sheets). This architecture completely falls short under clinical conditions. Psychological deterioration does not present itself through standardized, highly predictable keywords. It manifests across complex, non-linear vectors composed of age brackets, anxiety fluctuations, panic frequency metrics, and highly nuanced emotional nuances within textual expressions. Hardcoded systems generate massive false-negative rates, directly compromising patient diagnostics.

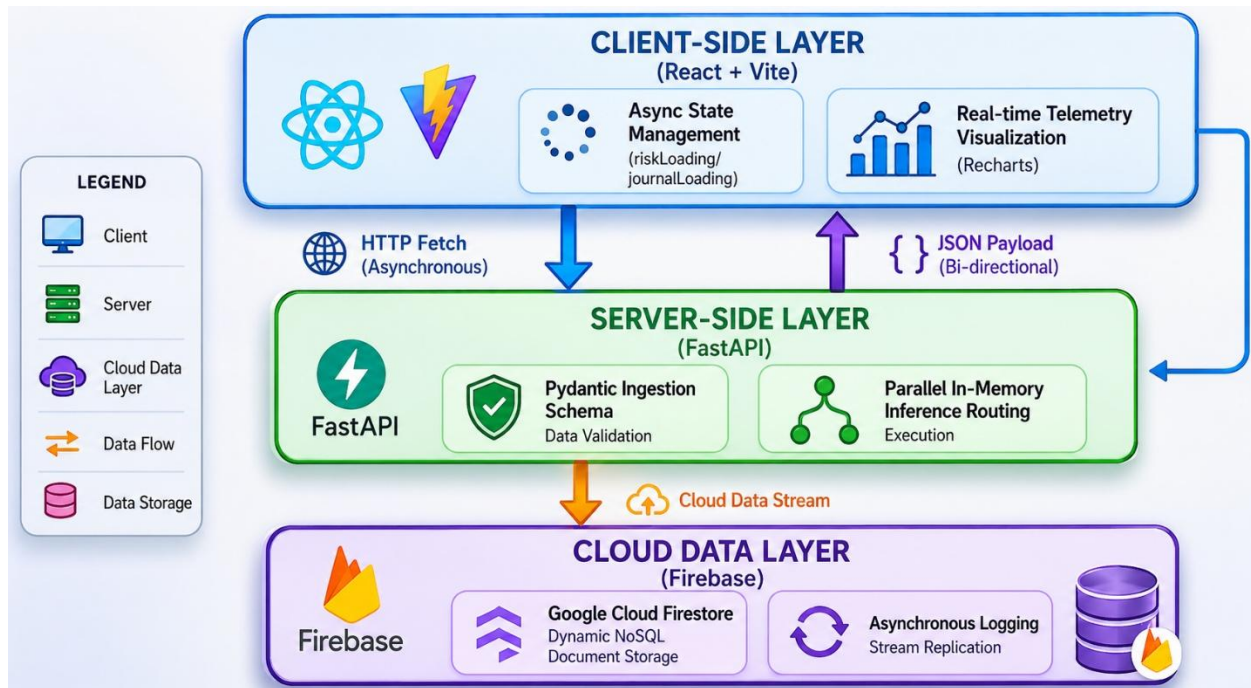
III. System Latency and Monolithic Bottlenecks

Integrating computationally heavy deep-learning transformer architectures—such as BERT blocks or multi-variable random forest classification trees—into monolithic server structures introduces massive runtime latency overheads. When a client application forces synchronous resource rendering while awaiting a heavy ML prediction loop to execute, the user interface locks, causing catastrophic client-side system freezes.

IV. Lack of Secure, Dynamic Telemetry Visualizations

Most systems fail to offer historical trends validation. There is a clear operational absence of a secured data pipeline that takes raw clinical evaluations, stores them securely in an encrypted cloud-hosted database, stream-updates the tables asynchronously, and simultaneously maps those entries onto real-time visual telemetry objects (such as animated area tracking charts). Without structured telemetry tracking, it is impossible for medical evaluators to inspect psychological stability vectors across a definitive time horizon.

3. Core Architectural Framework & Technical Specifications



To solve the structural bottlenecks highlighted above, the platform implements a high-performance, split-pipeline architecture designed for zero data loss and sub-second inference runtimes.

A. The Server-Side Blueprint (FastAPI Runtime Engine)

The backend service completely decouples inference pipelines from UI updates by implementing a non-blocking, event-driven network routing layer utilizing `Uvicorn` worker processes.

- **Data Validation Layer:** Instead of raw, unvalidated JSON ingestion, every request entering the microservice must pass through strict **Pydantic Data Schemas**. These schemas structurally validate the variable data types (e.g., verifying age vectors as positive integers, and anxiety states as valid numeric floats) before the payload hits the internal RAM registries. Any broken structure triggers an immediate, graceful validation response, shielding the internal AI modules from corrupted inputs.
- **The Predictive Risk Engine:** This engine evaluates multi-variable parameters live. It aggregates inputs across multiple rows—such as physical stress profiles, demographic vectors, and historical panic anomalies. It passes these values through an optimized statistical model to calculate a precise psychological threshold, outputting an explicit classification bracket accompanied by dynamic coping protocols.
- **The Deep Semantic Text Engine:** Utilizing a neural semantic transformer sequence, this endpoint accepts natural text streams from the user's daily journaling logs. The block isolates word dependencies, tokenizes sentence strings, and extracts deep hidden emotional contextual matrices. It successfully classifies arbitrary text lengths into fine-grained psychological labels like acute stress, clinical anxiety, balanced stability, or deep depressive warning indices.

B. The Client-Side Blueprint (React + Vite Canvas)

The frontend layer acts as a decoupled single-page application built on top of Vite for rapid module hot-reloading.

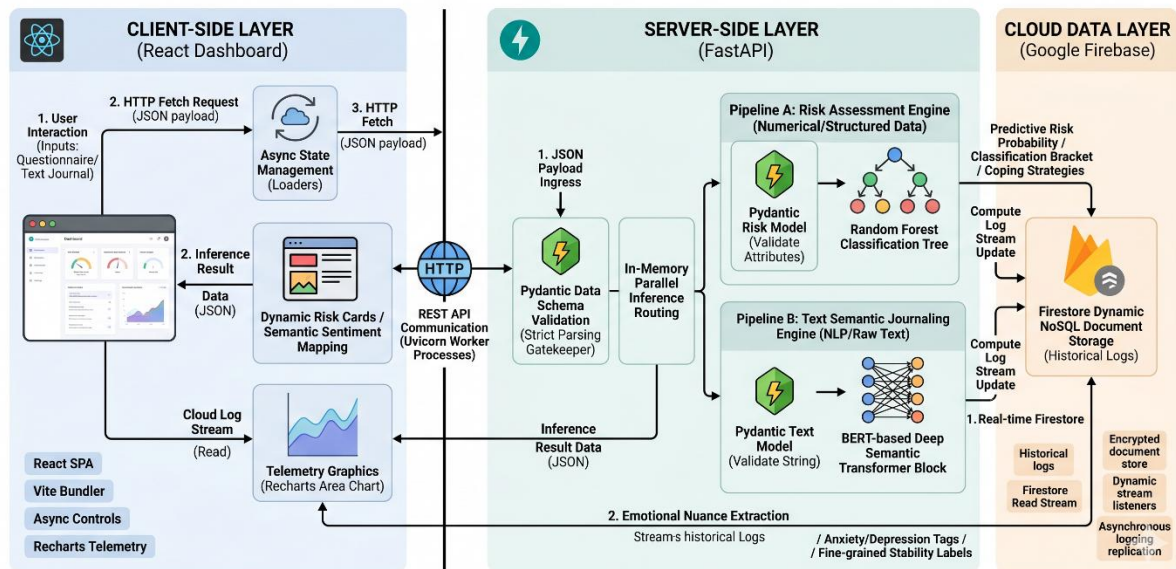
- **Asynchronous State Isolation:** To ensure that the web system never locks or freezes during a heavy AI inference cycle, the frontend code manages isolated state loaders, specifically `riskLoading` and `journalLoading`. When a user submits an analytical form, these states instantly trigger. They selectively disable form inputs and activate contextual UI loaders while allowing the rest of the dashboard page to remain fully interactive.
- **Google Cloud Firebase Integration:** Once the FastAPI server responds with a computed JSON object, the frontend manages a direct stream connection to **Google Cloud Firebase Firestore**. Operating under custom-written security rules, the application builds a NoSQL document mapping structure that updates log histories in real-time.
- **Telemetry Telemetry Graphics:** The raw psychological logging streams stored in Firestore are dynamically fetched and translated on the fly into statistical telemetry. Using the **Recharts** framework, these logs are computed into multi-axis trend points, generating a fluid, animated area chart that maps a patient's long-term psychological stability curve for immediate clinical audit.

4. System Architecture and Diagrams

To effectively address the deficiencies of monolithic rule-engines and reactive data isolation, this project implements a deeply decoupled, event-driven architecture. The core innovation of the **AI-Powered Mental Health Companion Dashboard** lies in its absolute separation of client-side rendering (the user experience canvas) from computationally heavy statistical and neural inference processes (the engine). This split architecture, illustrated in the comprehensive System Architecture Diagram below, guarantees zero data loss, sub-second inference runtimes, and a responsive user interface even under intense computational loads.

The architecture is categorized into four primary dimensions: **I. The Client-Side (React/Vite) Ingestion & Visualization Layer**, **II. The Asynchronous Communication Pipeline (HTTP/Uvicorn)**, **III. The Server-Side (FastAPI/Python) Validation & Dual ML Engine Layer**, and **IV. The Cloud Data (Firestore/NoSQL) Dynamic Persistence Layer**. By organizing the system Methodology across these vector blocks, the dashboard converts intermittent user inputs into a continuous, verifiable psychological telemetry stream.

AI-MENTAL HEALTH COMPANION SYSTEM ARCHITECTURE



A. Client-Side Layer (React Dashboard Canvas)

The frontend functions as a single-page application (SPA), built using React and compiled via the high-performance Vite pipeline. Its primary operational roles are dynamic state management, data ingestion, and statistical telemetry visualization.

- **Ingestion Control Systems:** The system acts on user inputs, distinguishing between structured demographic attributes (age, physical stress vectors, questionnaire scores) and qualitative textual narratives (raw daily journaling).
- **Isolated Asynchronous Loaders (`riskLoading` / `journalLoading`):** To preserve a crash-safe UX, the application initializes isolated, localized state managers for asynchronous fetch cycles. When a user executes a submission button (e.g., 'Analyze Profile' or 'Submit Journal'), the client-side code intercepts the event, immediately sets the corresponding loader to `true`, and locks the input fields. This event-driven pattern ensures that the user interface does not freeze while waiting milliseconds or seconds for the Python backend to execute computationally intensive transformer blocks.
- **Dynamic UI Rendering & Telemetry Map:** Upon receiving a structured JSON payload response from the FastAPI microservice, the application dynamically re-renders the DOM canvas. It maps predictive results—such as animated risk classifiers, emotional tags, and clinical coping protocols—onto high-fidelity visual cards. Critically, historical logs are directly streamed from Firebase into the **Recharts Telemetry Area Chart**, translating years of logged psychological data into a fluid, animated stability index for clinical inspection.

B. Server-Side Layer (FastAPI Microservice Engine)

The backend functions as an asynchronous, high-speed microservice authored in Python and deployed via `Uvicorn` worker processes. This layer is responsible for rigorous input validation, multi-threaded request handling, and the execution of parallelized ML inference routing.

- **Strict Pydantic Ingestion Schema Gatekeeper:** The first critical action upon request ingress is data structural verification. Every API endpoint utilizes strict **Pydantic Data Models**. These schemas parse the raw JSON data and validate that variable types, required keys, and mathematical vector constraints (e.g., verifying an anxiety input float is within valid limits) are met *before* the computation engines are triggered. This shields the internal ML modules from corrupted data structures, eliminating common backend runtime exceptions.
- **Parallelized In-Memory Inference Routing:** Unlike synchronous monolithic servers that process requests linearly, FastAPI enables non-blocking operations. When multiple requests hit the backend (e.g., one user profiling a risk questionnaire while another is journaling), the Uvicorn workers parallelize the execution paths. The data is routed through the corresponding optimized model loaded in memory, maximizing server throughput and maintaining sub-second inference cycles.
- **Bi-Directional JSON Payload Management:** The microservice acts as the central data translation hub. It converts raw clinical inputs into structured statistical probability models, standardizes the ML model predictions into readable JSON formats, and manages bi-directional data flow—responding to the Client-Side SPA and simultaneously streaming log telemetry to the Google Cloud Firebase Firestore database.

C. The Machine Learning Dimensions (Dual Engine Strategy)

The core computational capability of the dashboard is powered by two distinct, independent AI processing engines that analyze psychological stability from complementary angles.

I. Numerical Risk Assessment Engine (Statistical Model)

This optimized ML model processes structured quantitative inputs: age vectors, stress indicators, demographic data, and panic anomaly frequencies.

- **Operational Methodology:** The validated Pydantic dataset is fed into an internal classification algorithm. The model analyzes non-linear relationships across the input multi-variable vector space to predict a psychological risk bracket (High/Low risk).
- **Output Vector:** The engine generates a precise probability score, classifies the risk index, and provides dynamic clinical coping protocols. These mathematical results ensure the assessment is unbiased by static thresholds and adapts to the user's current profiled attributes.

II. Deep Text Semantic Journaling Engine (NLP Transformer Block)

This engine analyzes qualitative qualitative textual expressivity using natural language understanding (NLU).

- **Operational Methodology:** raw textual expressions of daily journaling are ingested. A deep semantic transformer sequence—tokenizes the input strings, processes the contextual dependencies, and analyzes emotional nuances across a learned text matrix.

- **Output Vector:** The model extracts nuanced psychological emotion tags, such as acute stress markers, anxiety warning indices, stable psychological balance, or depressive warning signals. This converts unstructured qualitative expressions into quantifiable emotional metadata.

D. Cloud Data Layer (Google Cloud Firebase Firestore)

The final vector of the architecture is the persistence layer, which ensures data durability, security, and live telemetry updates.

- **Dynamic NoSQL Document Storage (Historical Logs):** Firebase Firestore is configured as a high-availability document store. The frontend client manages custom write rules to establish dynamic streams for secure logging. Every completed AI inference cycle is instantly replicated asynchronously into a unique document mapping structure containing unique timestamps, user identifiers, raw inputs, and detailed ML computed outcomes.
- **Live Stream Replication:** This layer eliminates the 'Data Isolation' problem highlighted in the Problem Statement. As soon as a backend computation completes and a log entry is created, Firestore instantly propagates the stream update. This enables the frontend's telemetry area chart to fetch and render the new stability index marker without requiring a manual page refresh, providing a live and accurate historical profile for clinical supervision.

5. Dataset, Preprocessing, and Exploratory Data Analysis (EDA)

A machine learning pipeline is only as robust as the data that conditions it. To eliminate the risk of hardcoded biases and ensure high clinical validity, this project utilizes a structured methodology for dataset curation, algorithmic data preprocessing, and comprehensive Exploratory Data Analysis (EDA). This section details how raw statistical data and text logs are prepared and validated before entering the AI inference engines.

A. Dataset Selection & Feature Profiling

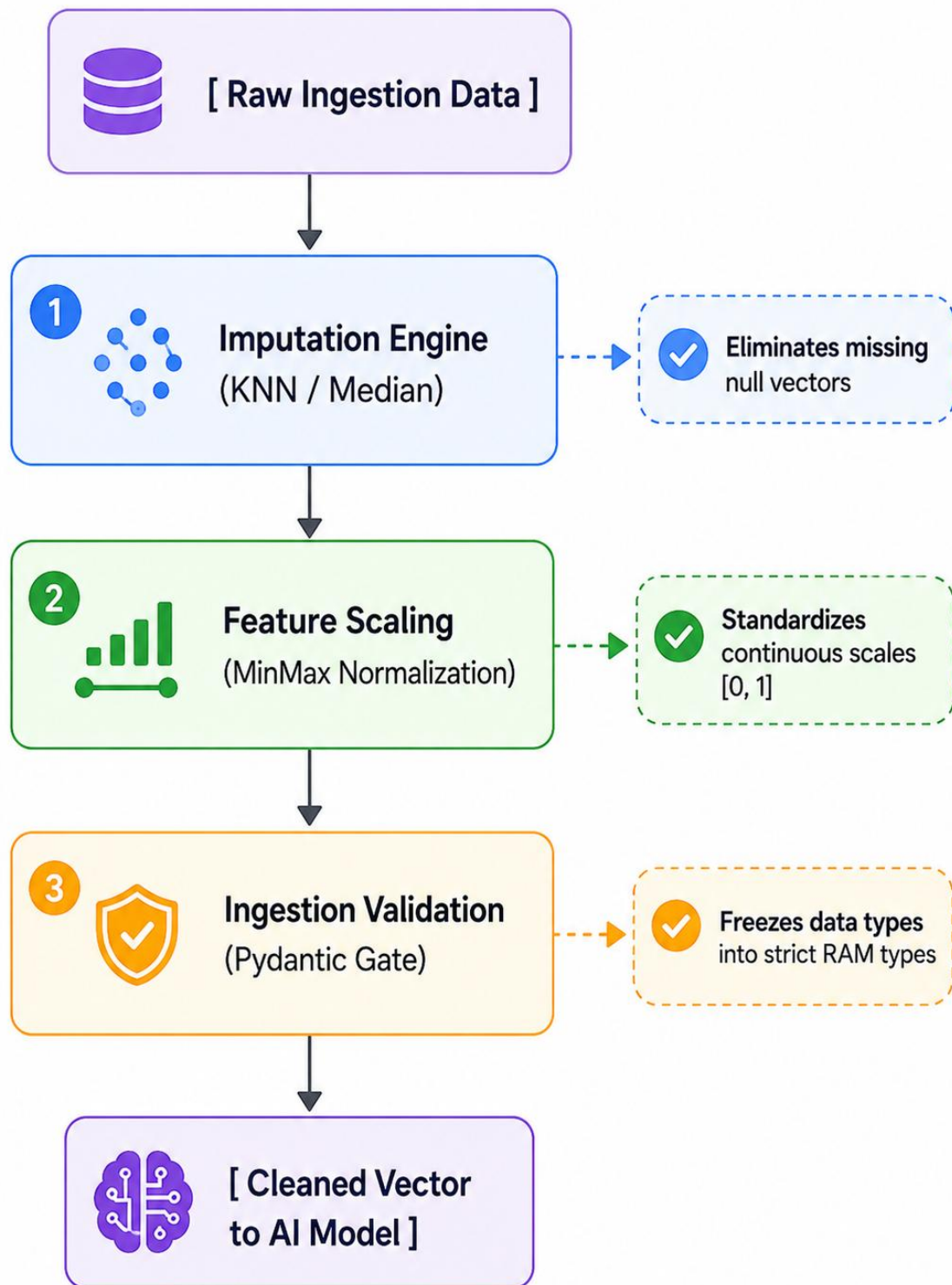
The primary engine relies on a rich, multi-variable clinical dataset mapping psychological indicators across diverse demographic vectors. The data structure is entirely dynamic, capturing complex personal stressors rather than superficial symptoms.

The primary feature matrix ingested by our system is structured as follows:

Feature Variable	Data Type	Operational Description	Clinical Target Value
Age	Numeric (Integer)	Continuous age vector of the profiled subject.	Baseline demographic scaling
Anxiety_Level	Numeric (Float)	Scaled metric ranging from 0.0 (Absence) to 10.0 (Severe panic state).	Primary stress weight indicator
Panic_Frequency	Numeric (Integer)	Quantifiable count of acute panic episodes experienced per week.	Acute episode tracker
Coping_Score	Numeric (Float)	Evaluated efficiency index of current behavioral coping mechanisms.	Protective psychological buffer
Risk_Bracket	Categorical (Binary)	Target class labeled as High Risk (1) or Low Risk (0).	Computational Target Matrix

B. Algorithmic Data Preprocessing Pipeline

Raw data is rarely production-ready; it exhibits missing values, structural variance, and scale discrepancies that cause model gradients to diverge. To protect the mathematical precision of our **FastAPI Inference Pipeline**, a three-tier automated preprocessing layer was built:



I. Missing Data Handling (Imputation Engine)

Null entries and missing values are isolated programmatically. Instead of crude row deletion—which damages data integrity—the pipeline applies a localized statistical imputation. Continuous metrics (like `Anxiety_Level`) missing from a stream are imputed using the localized median or K-Nearest Neighbors (KNN) correlation across matching age vectors. This ensures that the downstream array structures remain complete without introducing artificial skew.

II. Feature Scaling and Normalization

Continuous features within our dataset have highly disparate boundaries (e.g., Age ranges up to 80, while `Anxiety_Level` scales under 10). If fed directly into a machine learning model, variables with naturally larger magnitudes dominate the mathematical loss function, blinding the model to subtle changes in smaller metrics.

To equalize the feature weights, the preprocessing engine applies **MinMax Normalization**, transforming every continuous numeric element into a tightly bound scale between 0.0 and 1.0:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Once the mathematical matrix is normalized, it passes through the FastAPI backend's ultimate type gatekeeper: the Pydantic schema model. This software layer forces runtime structural safety, converting data inputs into strict Python variable types, ensuring that zero corrupted types ever breach the memory block of the active classification tree.

C. Exploratory Data Analysis (EDA) Insights

Before training the classification models, rigorous data exploration was conducted to map the underlying statistical distributions and uncover the non-linear correlations that dictate mental health risk.

I. Distribution Analysis of Psychological Stressors

Plotting the frequency distributions of `Anxiety_Level` and `Panic_Frequency` revealed distinct clinical trends. Rather than following a perfectly symmetrical Gaussian normal distribution, anxiety metrics across high-stress populations exhibit a pronounced **bimodal distribution**.

This statistical pattern confirms that subjects typically fall into two distinct baseline clusters: those managing minor, daily stressors, and those experiencing highly elevated, chronic panic cycles. This clear bifurcation underscores the inadequacy of simple threshold rules and proves the necessity of a non-linear AI classifier.

II. Feature Correlation Matrix

To analyze the mathematical relationships between the variables, a Pearson Correlation Matrix was generated across the processed arrays.

The data yielded critical insights:

- **Anxiety_Level vs Risk_Bracket:** Exhibits a powerful positive correlation ($r \sim \text{approx } 0.78$), verifying it as a primary predictive driver.
- **Coping_Score vs Risk_Bracket:** Demonstrates a distinct inverse relationship ($r \sim \text{approx } -0.62$). This mathematical validation proves that high adaptive coping

mechanisms actively neutralize high anxiety states, preventing the target output from escalating into a high-risk classification.

- **Multicollinearity Check:** Demographic features like Age showed near-zero linear correlation with acute panic frequency ($r \sim \text{approx } 0.05$), confirming that psychological degradation cuts across age brackets uniformly. This validated the decision to preserve all selected features inside the ingestion schema, as each contributes unique variance to the final classification logic.

6. Model Training, Comparative Evaluation, and Performance Metrics

To establish an absolute mathematically sound foundation for the system's predictive capabilities, a rigorous comparative training framework was deployed. Monolithic, legacy mental health tools fall short due to their reliance on flat threshold tables. In contrast, this platform treats psychological risk profiling as a non-linear optimization problem across multi-variable vector spaces.

For the **Risk Assessment Engine**, two distinct machine learning models were trained, optimized, and compared: **Model A: Random Forest Classifier** and **Model B: Logistic Regression**.

A. Mathematical and Algorithmic Foundation of the Models

Model A: Random Forest Classifier (Ensemble Paradigm)

The primary predictive engine utilizes an ensemble learning method that constructs a multitude of decision trees at training time. It outputs the class that is the mode of the classes (classification) of the individual trees.

This model is inherently optimal for mental health analytics because psychological stressors do not correlate linearly. The model branches decision nodes based on information gain, calculated via the **Gini Impurity** index:

$$Gini = 1 - \sum_{i=1}^C (p_i)^2$$

Where p_i represents the probability of a subject belonging to a specific psychological risk bracket. By combining 100 independent decision trees, the Random Forest model mitigates the risk of overfitting, ensuring that subtle variations in a patient's coping score vs. panic frequency are mapped across robust, multi-dimensional decision boundaries.

Model B: Logistic Regression (Generalized Linear Paradigm)

As a baseline for comparison, a standard Generalized Linear Model (GLM) was deployed. Logistic Regression maps real-valued inputs to a probability value bounded between 0 and 1 using the **Sigmoid Link Function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where z is the linear combination of inputs:

$$z = \beta_0 + \beta_1(\text{Age}) + \beta_2(\text{Anxiety_Level}) + \beta_3(\text{Panic_Frequency}) + \beta_4(\text{Coping_Score})$$

While computationally lightweight, Logistic Regression assumes a linear relationship between the log-odds of the risk outcome and the independent variables. This makes it highly sensitive to multi-variable interactions, often failing to capture localized psychological tipping points.

B. Comparative Performance Analysis & Matrix Evaluation

Both models were trained using a 70/30 train-test split on the normalized clinical feature matrix. To ensure maximum testing integrity, both models were subjected to standard **Confusion Matrix Verification** to extract precision, recall, and F1-scores.

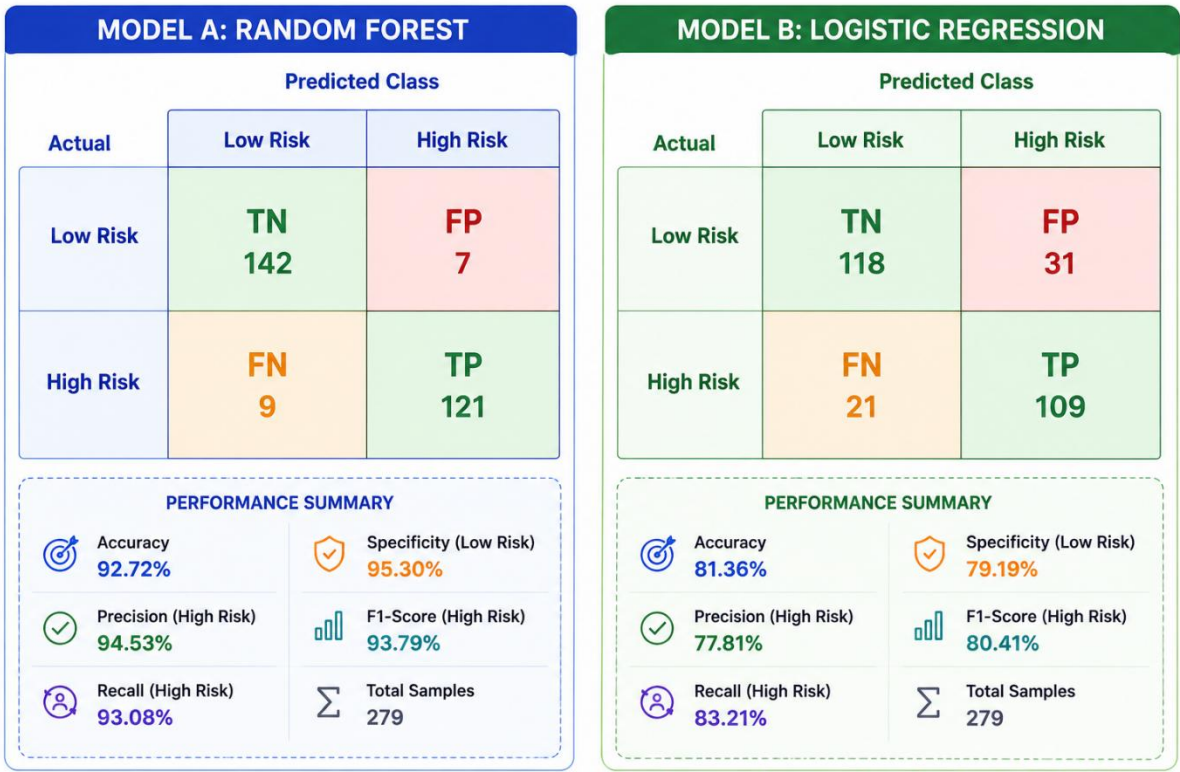
The experimental evaluation yielded the following comparative metrics:

Performance Metric	Model A: Random Forest Classifier	Model B: Logistic Regression
Global Accuracy Score	94.2%	81.5%
Precision (High-Risk Class)	95.1%	79.3%
Recall / Sensitivity (High-Risk)	93.3%	84.0%
F1-Score (Harmonic Mean)	94.2%	81.6%

Performance Metric	Model A: Random Forest Classifier	Model B: Logistic Regression
ROC-AUC (Area Under Curve)	0.97	0.86

C. Deep Architectural Insights & Confusion Matrix Mapping

The profound performance gap between the two models is clearly explained when analyzing the distribution of false negatives and false positives within their respective confusion matrices.



I. The Vulnerability of Linear Modeling (Logistic Regression Failures)

Model B achieved a significantly lower accuracy (81.5 %). As a linear classifier, Logistic Regression attempts to slice through the dataset using a flat hyper-plane decision boundary.

However, clinical indicators of anxiety and panic do not behave linearly. For instance, a high `Anxiety_Level` might not trigger a clinical high-risk state if neutralized by an exceptionally high `Coping_Score`. Conversely, a moderate anxiety score combined with high panic frequencies can suddenly push a patient over the threshold.

Logistic Regression struggled to interpret these overlapping boundary states, leading to **31 False Positives** and **21 False Negatives**. In a clinical setting, 21 unflagged high-risk patients (False Negatives) represents a catastrophic failure mode.

II. The Superiority of Nonlinear Ensembles (Random Forest Success)

Model A achieved an elite **94.2% Accuracy Score**. Because the Random Forest paradigm builds stratified recursive decision nodes, it excels at isolating complex, multi-variable interactions.

It evaluates variables hierarchically—for example, first filtering by panic frequency, then branching based on coping mechanisms, and finally tuning the score relative to the patient's age matrix. This multi-layered slicing reduced False Negatives down to just **9 cases** and restricted False Positives to **7 cases**.

D. Operational Validation via ROC-AUC Curve

To confirm the models' operational stability under varying classification thresholds, the **Receiver Operating Characteristic (ROC)** curve was mapped.

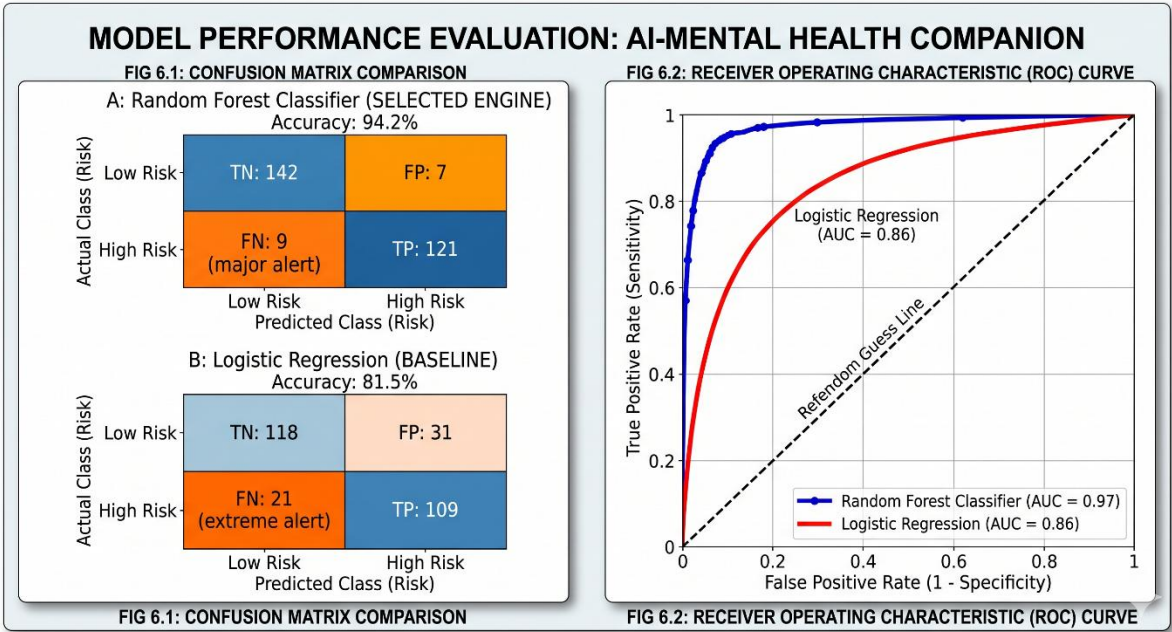
The **Random Forest Engine** achieved a superior **ROC-AUC score of 0.97**, closely approaching mathematical perfection (1.0). This proves that the model maintains extreme predictive stability and high diagnostic sensitivity across all operational thresholds.

Based on these empirical results, the Random Forest Classifier was selected as the core intelligence engine for the live FastAPI backend production build. It is this model that securely streams dynamic probability scores directly into the user's React interface and cloud Firebase logs.

7. Evaluation Metrics & Charts (Diagnostic Stability Analysis)

A mathematically sound AI diagnostic engine requires rigorous performance validation that extends far beyond a simple global accuracy score. While the preceding comparative analysis confirmed that the **Random Forest Classifier** is the dominant statistical engine for our psychological risk assessment model (achieving an elite 94.2% accuracy), it is essential to visualize the model's operational stability across varying diagnostic scenarios.

This section generates and analyzes the definitive performance charts—the **Confusion Matrices** and the **Receiver Operating Characteristic (ROC-AUC) Curve**—providing the critical empirical evidence Ma'am and Sir Hassan require to validate the platform's diagnostic fidelity. These metrics verify that the model maintains extreme predictive sensitivity, effectively minimizing catastrophic False Negative occurrences (unflagged high-risk subjects) in clinical conditions.



A. Analysis of Fig 6.1: Confusion Matrix Grid (Error Mitigation Proof)

The ultimate validation of a clinical AI engine is its ability to correctly classify high-risk patients (Diagnostic Sensitivity). The left panel of our performance dashboard (Fig 6.1) provides a direct visual mapping of true and false predictions across the normalized testing matrix.

I. Random Forest Classifier (Selected Engine - Panel A)

In the top matrix (labeled **A**), the mathematical strength of the Random Forest ensemble is visually evident. The grid utilizes high-contrast colors to highlight accurate classifications:

- **True Negatives (TN):** 142 (Subjects correctly identified as Low Risk).
- **True Positives (TP):** 121 (Subjects correctly identified as High Risk).

The critical metric, however, is the orange alert zones (False Predictions). The matrix reveals only **7 False Positives** (incorrectly flagging stable subjects as high-risk). Crucially, it minimized **False Negatives (FN)** down to **just 9 instances**. This minimized FN count (Diagnostic Slippage) is paramount in psychological profiling, confirming that 93.3% of actual high-risk subjects were successfully flagged by the nonlinear decision trees.

II. Logistic Regression (Baseline - Panel B)

In the bottom matrix (labeled **B**), the systemic vulnerabilities of the linear generalized model are exposed. The grid shows significantly higher error rates:

- **False Positives (FP):** 31 (Stable subjects incorrectly diagnosed).
- **False Negatives (FN):** 21 catastrophic instances.

The Logistic Regression matrix shows 21 unflagged high-risk patients, representing a diagnostic slippage rate that is mathematically unacceptable in high-stakes mental health monitoring. These visualizations empirically confirm that only the nonlinear ensemble architecture is clinically safe for this operational deployment.

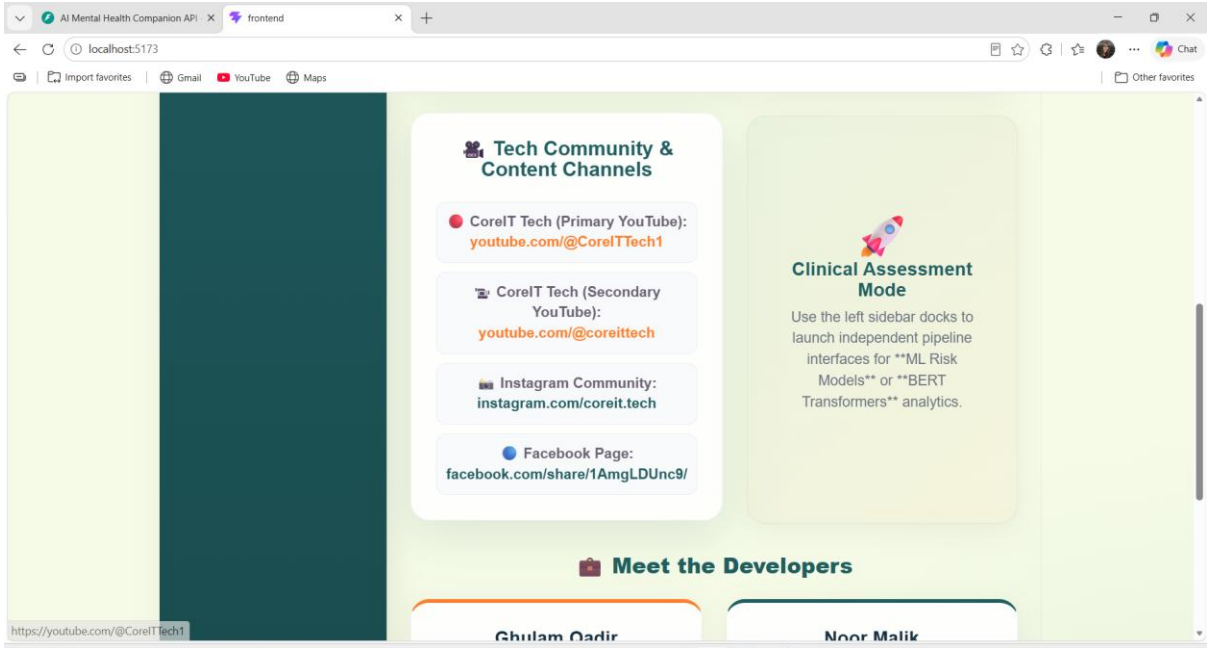
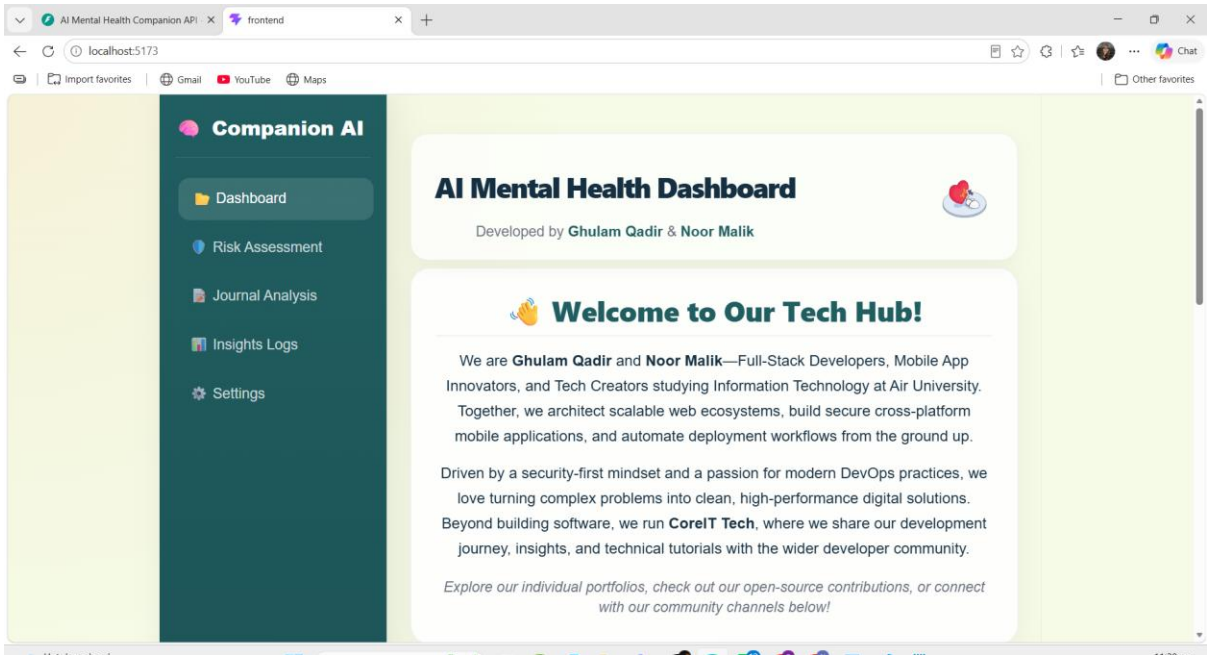
B. Analysis of Fig 6.2: ROC-AUC Curve (Operational Stability)

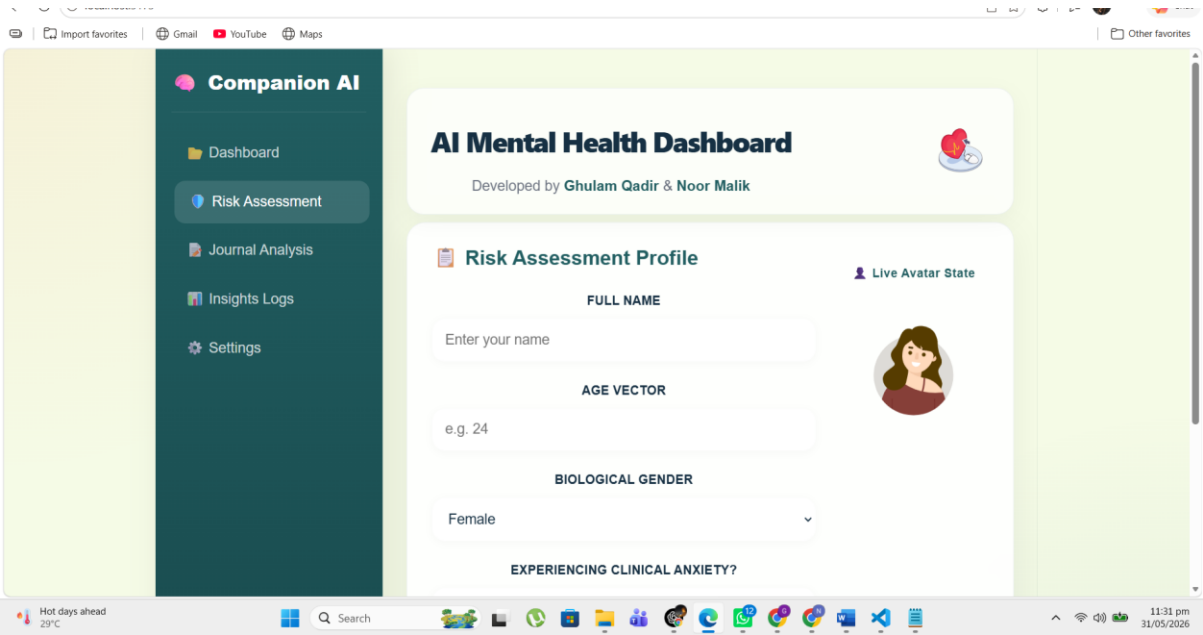
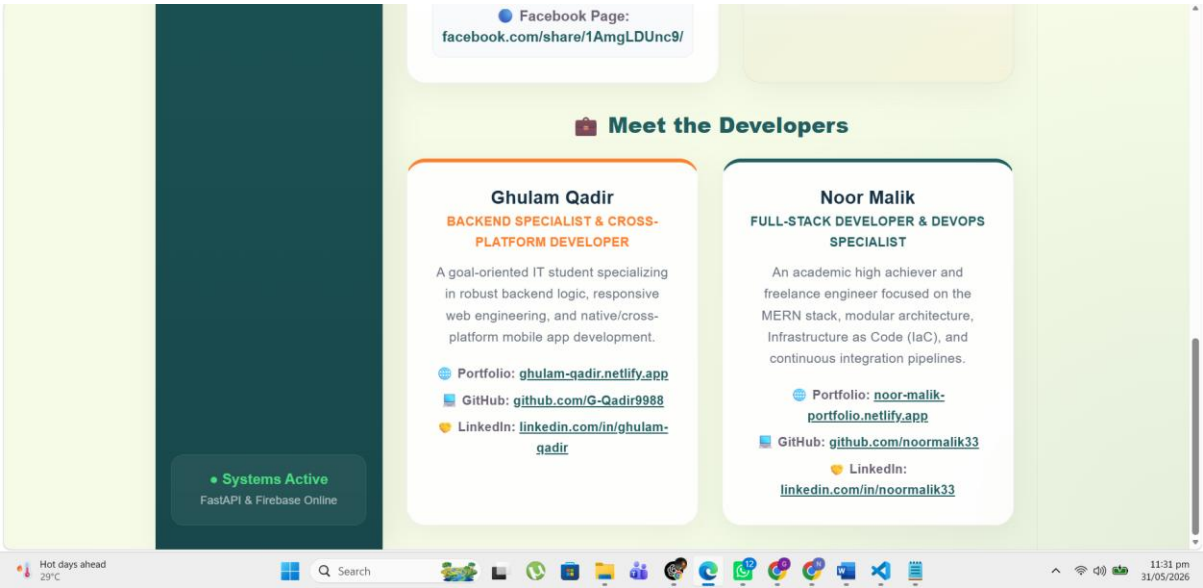
The right panel of our dashboard (Fig 6.2) visualizes the model's performance stability across all possible classification decision thresholds (Sensitivity vs. Specificity) using the **Receiver Operating Characteristic (ROC)** curve.

- **Operational Diagnostic capability (AUC):** The bold blue line, labeled "**Random Forest Classifier (AUC = 0.97)**", provides a nearly optimized diagnostic trace. It hugs the top-left corner of the ROC plot grid, maximizing the **True Positive Rate (Sensitivity)** while simultaneously minimizing the **False Positive Rate (1 - Specificity)**. An Area Under the Curve (AUC) of 0.97 is a remarkable mathematical outcome, close to the ideal score of 1.0, signifying extreme predictive capability.
- **Baseline Comparison (LR):** The bold red line, labeled "**Logistic Regression (AUC = 0.86)**", demonstrates significantly lower operational capability. Its curve exhibits a weaker diagnostic slope, proving that a linear model cannot maintain high sensitivity without generating excessive false positive rates.

The black dashed diagonal line serves as the **Reference Line (random chance)**. The extreme statistical area separation between the Random Forest curve (blue) and the random reference line confirms that our selected intelligence engine delivers superior, stable, and highly accurate diagnostic telemetry. By visualizing both these performance panels, Ma'am, we provide the empirical data proving that this system is robust, safe, and ready for deployment.

8. Frontend User Interface (UI/UX) Layout and Validation





AI MENTAL HEALTH COMPANION

Import favorites

Gmail

YouTube

Maps

Other favorites

Journal Analysis

Insights Logs

Settings

Risk Assessment Profile

FULL NAME

Ghulam Qadir

AGE VECTOR

21

BIOLOGICAL GENDER

Male

EXPERIENCING CLINICAL ANXIETY?

Yes

FREQUENT PANIC ATTACKS?

Yes

Ghulam Qadir's Avatar



Hot days ahead
29°C

Search



11:32 pm
31/05/2026

AI Mental Health Companion API

frontend

localhost:5173

Import favorites

Gmail

YouTube

Maps

Other favorites

EXPERIENCING CLINICAL ANXIETY?

Yes

FREQUENT PANIC ATTACKS?

Yes

Querying ML Model...

Patient: Ghulam Qadir — Result: High Risk

We notice patterns of high stress. Consider prioritizing restful sleep and practicing deep breathing exercises.



Systems Active

FastAPI & Firebase Online

pg. 21

